ReDISchool

Week 6

Introduction to Programming - Fall 2026

Lists & Tuples 

One **variable**. Many **values**.

Agenda

01	Recap & Questions
02	Lists – create, read, change
03	Loops with lists
–	Break (15 min)
04	Tuples
05	More list methods
06	Homework

Parts 02, 03, 04 and 05 have short exercises. ✍️

01 - Recap from last week

Last week we covered:

- 🔁 `for` **loops** – `for i in range(5):` repeats a known number of times
- ⏳ `while` **loops** – repeat until a condition changes
- ⏭️ `break` **&** `continue` – exit a loop early or skip an iteration
- 📋 **Lists (sneak peek)** – `names = ["Anna", "Bruno", "Carla"]`
- 🔍 **Patterns** – filter, accumulate, find the max

Today we go deep on **lists**, **tuples**, and using **loops** with them.

Questions?

02 - Lists 

Create, read and change

What Is a List?

Imagine storing five friends' names:

- Without a list → one variable per name
- With 50 friends, that's 50 variables
- What if you don't know how many there are?

A **list** holds many values in **one variable**, in order, inside square brackets `[ ]`.

- The values are called **items**
- `len()` tells you how many items there are

```
# Without a list
friend1 = "Anna"
friend2 = "Bruno"
friend3 = "Carla"

# With a list
friends = ["Anna", "Bruno", "Carla"]

print(friends)       # ['Anna', 'Bruno', 'Carla']
print(len(friends))  # 3
```

Creating a List

- Square brackets `[ ]` with **commas** between items
- Can hold strings, numbers, booleans – or a mix
- An **empty list** `[]` is valid – add items later
- `len()` counts the items

```
# A list of strings
fruits = ["apple", "banana", "cherry"]

# A list of numbers
scores = [12, 45, 7, 30, 18]

# Mixed types
mixed = ["Alice", 25, True]

# Empty list
cart = []

print(len(fruits))  # 3
print(len(cart))    # 0
```

Reading Items by Index

Each item has a position number called an **index**, starting at **0**.

- `fruits[0]` – first item
- `fruits[1]` – second item
- `fruits[-1]` – last item
- `fruits[-2]` – second to last

Asking for an index that doesn't exist → `IndexError`

```
fruits = ["apple", "banana", "cherry"]
#          index 0  index 1   index 2

print(fruits[0])   # apple
print(fruits[1])   # banana
print(fruits[2])   # cherry
print(fruits[-1])  # cherry - last item
print(fruits[-2])  # banana

# This crashes:
print(fruits[5])   # IndexError!
```

Changing a List

Lists are **mutable** – you can change them after you create them.

- `fruits[i] = value` – replace an item
- `.append(value)` – add to the end
- `.remove(value)` – remove by value
- `.pop()` – remove and return the last item
- `in` – check if an item is in the list

```
fruits = ["apple", "banana", "cherry"]

fruits[1] = "mango"      # replace banana
print(fruits)  # ['apple', 'mango', 'cherry']

fruits.append("orange")  # add to end
print(fruits)
# ['apple', 'mango', 'cherry', 'orange']

fruits.remove("mango")   # remove by value
print(fruits)  # ['apple', 'cherry', 'orange']

last = fruits.pop()      # remove last
print(last)    # orange

print("apple" in fruits)  # True
print("mango" in fruits)  # False
```

Exercise 1 – Shopping List · 8 min

✍️ Start with this list and do each step in order, **printing the list after each change**:

```
shopping = ["milk", "bread", "eggs"]
```

1. Add `"butter"` to the end
2. Replace `"bread"` with `"pasta"` (use its index)
3. Remove `"eggs"` from the list
4. Print how many items are left

Expected output:

```
['milk', 'bread', 'eggs', 'butter']
['milk', 'pasta', 'eggs', 'butter']
['milk', 'pasta', 'butter']
3
```

03 - Loops with lists 🔁

Visit every item, one by one

Looping Over a List

You can loop over a list with `for`, just like with `range()`.

- No `range()` needed – just the list itself
- The loop variable takes **each item** in turn
- The loop runs **once per item**
- Same `for … in …` pattern as before

```
fruits = ["apple", "banana", "cherry"]

for fruit in fruits:
    print("I like", fruit)
```

```
I like apple
I like banana
I like cherry
```

```
scores = [12, 45, 7, 30]

for score in scores:
    print(score)
```

Looping with the Index – `enumerate()`

Normally a loop gives you the **item only**. Sometimes you also need to know **which position** you are at.

`enumerate()` gives you **both** at the same time – the position number and the item – so you don't have to count yourself.

By default the count starts at `0`. Use `start=1` to count from `1`.

```
fruits = ["apple", "banana", "cherry"]

# With enumerate - position AND item
for i, fruit in enumerate(fruits):
    print(i, fruit)
```

```
0 apple
1 banana
2 cherry
```

```
# Start counting from 1
for i, fruit in enumerate(fruits, start=1):
    print(f"{i}. {fruit}")
```

```
1. apple
2. banana
3. cherry
```

Pattern 1 – Filter

Filtering means keeping only the items that match a condition.

1. Loop over **every** item
2. Use `if` to check the condition
3. Print (or collect) only the items that pass

```
scores = [12, 45, 7, 30, 18]

for score in scores:
    if score > 15:
        print(score)
```

```
45
30
18
```

Pattern 2 – Accumulate & Find the Max

Accumulating – add up all the items.

Finding the max – keep the biggest seen so far.

Both follow the same shape:

1. Create a variable **before** the loop
2. Update it **inside** the loop
3. Use it **after** the loop

```
scores = [12, 45, 7, 30, 18]

# Accumulate
total = 0
for score in scores:
    total = total + score
print(f"Total: {total}")      # 112
print(f"Average: {total / len(scores)}")  # 22.4

# Find the max
highest = scores[0]
for score in scores:
    if score > highest:
        highest = score
print(f"Highest: {highest}")  # 45
```

Exercise 2 – Score Analyser · 10 min

✍️ Write a program that:

```
scores = [8, 3, 15, 7, 21, 4, 9]
```

1. Prints every score **above 10**
2. Calculates and prints the **total** of all scores
3. Finds and prints the **highest** score

Expected output:

```
15
21
Total: 67
Highest: 21
```

Break (15 min) 

04 - Tuples 🔒

Lists that never change

What Is a Tuple?

A **tuple** is like a list – but you **cannot change it** after you create it.

- Uses round brackets `( )` instead of `[ ]`
- **Immutable** – fixed once created
- Indexing, `len()`, `in` and looping work the same
- Use a tuple when the values should **never change**

	List	Tuple
Syntax	`[ ]`	`( )`
Can change?	✅ Yes	❌ No

```
# List - can be changed
fruits = ["apple", "banana"]
fruits[0] = "mango"    # works fine

# Tuple - cannot be changed
point = (10, 20)
point[0] = 99          # TypeError!

# When to use a tuple
days = ("Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday",
        "Sunday")
location = (55.6761, 12.5683)  # lat, long
red = (255, 0, 0)              # RGB colour
```

Reading & Unpacking a Tuple

Reading works exactly like a list:

- `point[0]` – first item
- `point[-1]` – last item
- `len(point)` – number of items
- `in` – check if an item is there

Unpacking – give each item its own variable, in one line.

```
point = (10, 20)
print(point[0])    # 10
print(point[-1])   # 20
print(len(point))  # 2

# Unpacking
x, y = (10, 20)
print(x)  # 10
print(y)  # 20

# Any size
name, age, city = ("Anna", 28, "Copenhagen")
print(name)  # Anna
print(city)  # Copenhagen
```

Looping Over a Tuple

Looping works exactly the same as with lists.

You can **unpack** items directly in the loop header – very useful with a list of pairs.

```
days = ("Monday", "Tuesday", "Wednesday")

for day in days:
    print(day)

# Unpack inside the loop
people = [("Alice", 28), ("Bruno", 34), ("Carla", 22)]

for name, age in people:
    print(f"{name} is {age} years old.")
```

```
Alice is 28 years old.
Bruno is 34 years old.
Carla is 22 years old.
```

Exercise 3 – Weekly Temperatures · 10 min

✍️ Write a program that:

```
temperatures = (18, 21, 19, 24, 22, 17, 20)
days = ("Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday",
        "Sunday")
```

1. Prints each day and its temperature
2. Finds and prints the **highest** temperature
3. Prints the **average** (one decimal place)

Need a hint?

- `f"{value:.1f}"` shows one decimal place

Expected output:

```
Monday: 18°C
Tuesday: 21°C
Wednesday: 19°C
Thursday: 24°C
Friday: 22°C
Saturday: 17°C
Sunday: 20°C
Highest: 24°C
Average: 20.1°C
```

05 - More list methods 

Sorting, searching, slicing and merging

More List Methods

Sorting

- `.sort()` – sorts the list **in place** (changes it)
- `.sort(reverse=True)` – sorts largest first
- `sorted(my_list)` – returns a sorted **copy**, the original stays the same

Other useful methods

- `.insert(i, value)` – add at a specific position
- `.index(value)` – find the position of an item
- `.count(value)` – how many times an item appears
- `.clear()` – remove all items

```
scores = [30, 12, 45, 7, 18]
scores.sort()
print(scores)    # [7, 12, 18, 30, 45]
scores.sort(reverse=True)
print(scores)    # [45, 30, 18, 12, 7]

# sorted() - keeps the original
original = [30, 12, 45, 7, 18]
copy = sorted(original)
print(original)  # [30, 12, 45, 7, 18]
print(copy)      # [7, 12, 18, 30, 45]

names = ["Anna", "Bruno", "Carla"]
names.insert(1, "Dina")
print(names)  # ['Anna', 'Dina', 'Bruno', 'Carla']
print(names.index("Bruno"))  # 2
print(names.count("Anna"))   # 1
```

Slicing a List

Just like strings, you can **slice** a list to get a section of it.

- `scores[start:stop]` – from `start` up to (not including) `stop`
- `scores[:3]` – from the beginning up to index 3
- `scores[2:]` – from index 2 to the end
- `scores[-2:]` – the last 2 items

Slicing gives a **new** list – the original is unchanged.

```
scores = [10, 20, 30, 40, 50]

print(scores[1:4])  # [20, 30, 40]
print(scores[:3])   # [10, 20, 30]
print(scores[2:])   # [30, 40, 50]
print(scores[-2:])  # [40, 50]

# Sorted + sliced = top 3
scores.sort(reverse=True)
print(scores[:3])   # [50, 40, 30]
```

Exercise 4 – Top Scores · 10 min

✍️ Write a program that:

```
scores = [42, 17, 89, 55, 23, 76, 34, 91, 60, 11]
```

1. Sorts the scores from **highest to lowest**
2. Prints the **top 3** scores (use slicing)
3. Prints the **lowest** score
4. Prints how many scores are **above 50**

Expected output:

```
Top 3: [91, 89, 76]
Lowest: 11
Scores above 50: 5
```

Merging Two Sorted Lists

We have two lists that are **already sorted**. We want **one** sorted list with every item — without sorting again.

The idea:

1. Look at the **front** item of each list
2. Take the **smaller** one and add it to the result
3. Move forward in the list you took it from
4. When one list runs out, add **the rest** of the other

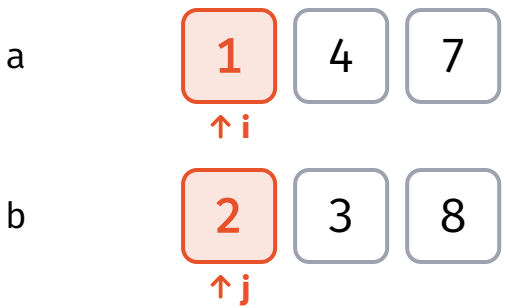
```
a = [1, 4, 7]
b = [2, 3, 8]

# Goal:
# [1, 2, 3, 4, 7, 8]
```

We remember our place in each list with an **index**:

- `i` is our position in `a`
- `j` is our position in `b`
- Both start at `0`

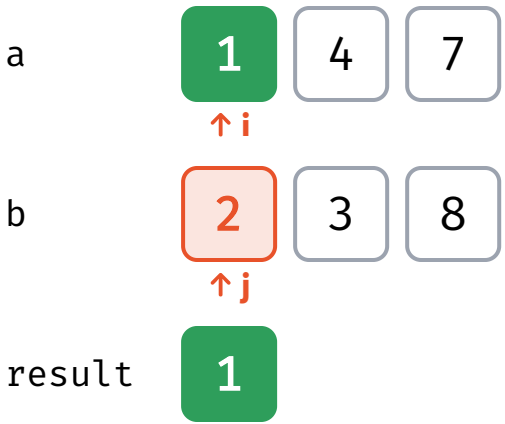
Merging Step by Step – Start



- `i = 0` points at `a[0]`, which is `1`
- `j = 0` points at `b[0]`, which is `2`
- `result` starts as an empty list

Orange = the front item of each list
Green = the item we take
Faded = already used

Merging Step by Step – Step 1 of 6

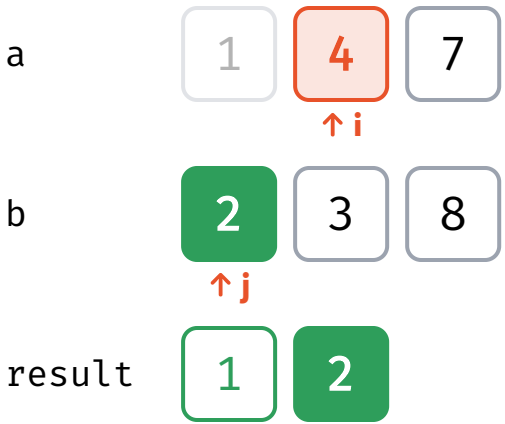


Compare `a[0]` and `b[0]`: `1` vs `2`

`1` is smaller → take it from `a`

```
result.append(a[i])
i = i + 1  # i is now 1
```

Merging Step by Step – Step 2 of 6

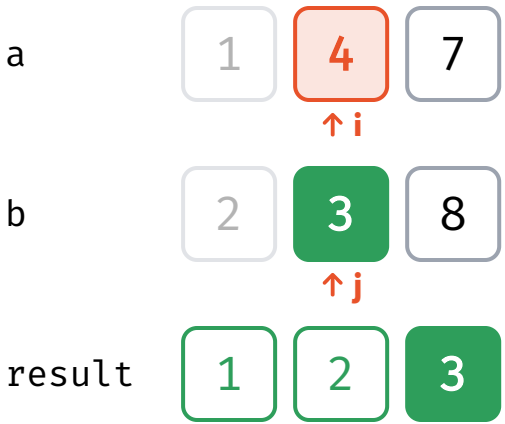


Compare `a[1]` and `b[0]`: `4` vs `2`

`2` is smaller → take it from `b`

```
result.append(b[j])
j = j + 1  # j is now 1
```

Merging Step by Step – Step 3 of 6

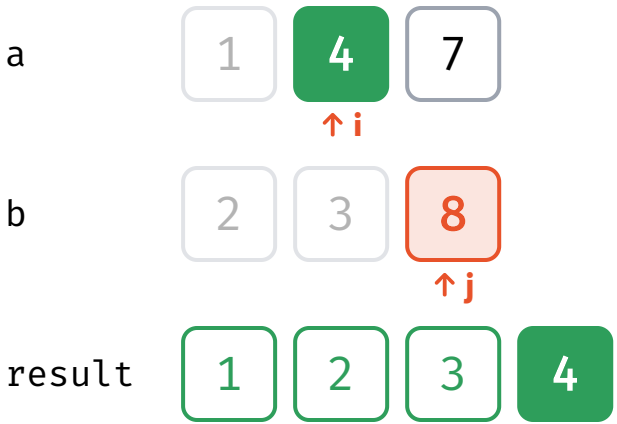


Compare `a[1]` and `b[1]`: `4` vs `3`

`3` is smaller → take it from `b`

```
result.append(b[j])
j = j + 1  # j is now 2
```

Merging Step by Step – Step 4 of 6

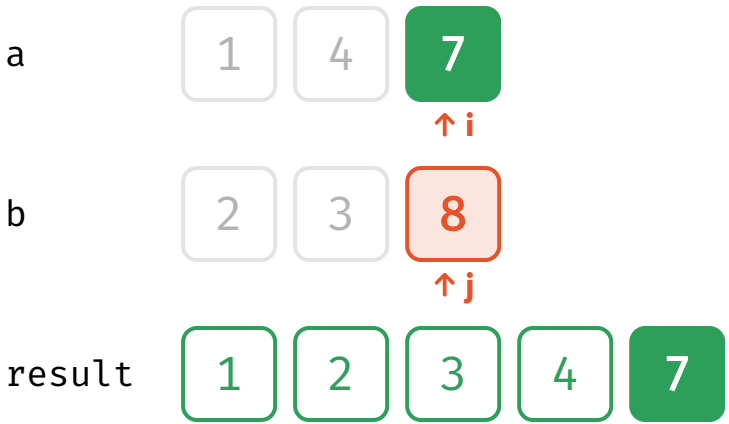


Compare `a[1]` and `b[2]`: `4` vs `8`

`4` is smaller → take it from `a`

```
result.append(a[i])
i = i + 1  # i is now 2
```

Merging Step by Step – Step 5 of 6



Compare `a[2]` and `b[2]`: `7` vs `8`

`7` is smaller → take it from `a`

```
result.append(a[i])
i = i + 1  # i is now 3
```

`i` is now `3`, which is `len(a)`. `a` is **used up**, so we stop comparing.

Merging Step by Step – Step 6 of 6



Add the rest. `b` is sorted, so everything left in it can go straight to the end.

`b[j:]` is `b[2:]`, which is `[8]`

```
for item in b[j:]:
    result.append(item)
```

Done! `[1, 2, 3, 4, 7, 8]` ✅

Merging Two Sorted Lists – The Code

```
result = []
i = 0  # position in a
j = 0  # position in b

# Compare front items while both lists have some left
while i < len(a) and j < len(b):
    if a[i] <= b[j]:
        result.append(a[i])
        i = i + 1
    else:
        result.append(b[j])
        j = j + 1

# One list is used up: add the rest of the other
for item in a[i:]:
    result.append(item)
for item in b[j:]:
    result.append(item)
print(result)  # [1, 2, 3, 4, 7, 8]
```

- Same `a` and `b` as before.
- We use `while`, not `for`: `i` and `j` move at **different** times.
- The `while` loop stops as soon as **one** list is used up.
- After the loop, one of `a[i:]` and `b[j:]` is always **empty**, so only one `for` loop adds anything.

Common Mistakes (1/2)

1. Index starts at 0, not 1

```
fruits = ["apple", "banana", "cherry"]
print(fruits[1])  # banana - NOT apple
```

2. Index out of range

```
print(fruits[5])  # IndexError!
# Only 0, 1, 2 exist
```

3. Trying to change a tuple

```
point = (10, 20)
point[0] = 99  # TypeError!
```

Common Mistakes (2/2)

4. `append()` returns `None` — don't assign it

```
cart = []
cart = cart.append("milk")  # WRONG
print(cart)  # None!

cart = []
cart.append("milk")         # RIGHT
print(cart)  # ['milk']
```

5. Changing a list while looping over it

```
# WRONG - unpredictable
for n in numbers:
    numbers.remove(n)

# RIGHT - collect into a new list
result = []
for n in numbers:
    if n % 2 != 0:
        result.append(n)
```

Mini Project – Student Report 🛠️

✍️ For each student:

1. Calculate their **average** score
2. Print their name, average, and whether they **passed** (average ≥ 60)

At the end, print the name of the student with the **highest** average.

```
students = [
    ("Alice", [85, 92, 78]),
    ("Bruno", [60, 55, 70]),
    ("Carla", [95, 88, 91]),
    ("Dina",  [40, 52, 48]),
]
```

Expected output:

```
Alice  – average: 85.0 – Passed
Bruno  – average: 61.7 – Passed
Carla  – average: 91.3 – Passed
Dina   – average: 46.7 – Did not pass
Top student: Carla
```

Need a hint?

- `f"{name:<6}"` pads the name to 6 characters

What We Learned Today

✅ A **list** `[ ]` holds many values in order – **mutable**, so you can change it

✅ Items have an **index** starting at `0`. Use `-1` for the last item

✅ `.append()`, `.remove()`, `.pop()` add and remove items

✅ Loop over a list with `for item in my_list:` – no `range()` needed

✅ `enumerate()` gives you the index and the item together

✅ Patterns: filter, accumulate, find the max – create **before**, update **inside**, use **after**

✅ A **tuple** `( )` is like a list but **immutable** – for values that should never change

✅ `.sort()` sorts in place, `sorted()` returns a sorted copy

✅ Slicing `my_list[1:4]` gives you a section of the list

Questions?

06 - Homework! 

📌 Commit your work to GitHub when you're done!

Homework 1 – Favourite Foods

🏠 Create a list of **5 foods** you like. Loop through it and print each one with its position number, **starting from 1**.

Expected output (with your own foods):

```
1. pizza
2. sushi
3. pasta
4. tacos
5. soup
```

Homework 2 – Sum of a List

🏠 Use a loop to calculate the sum of all numbers in this list. Do **not** use the built-in `sum()` function.

```
numbers = [4, 8, 15, 16, 23, 42]
```

Expected output:

```
108
```

Homework 3 – Count the Long Words

🏠 Loop through the list and count how many words have **more than 4 letters**. Print the count.

```
words = ["cat", "elephant", "dog",
         "tiger", "ox", "penguin"]
```

Expected output:

```
3
```

Homework 4 – Tuple Unpacking

🏠 Loop through the list of `(name, age)` tuples and print a sentence for each person.

```
people = [("Alice", 28), ("Bruno", 34),
          ("Carla", 22)]
```

Expected output:

```
Alice is 28 years old.
Bruno is 34 years old.
Carla is 22 years old.
```

Homework 5 – Reverse a List

🏠 Without using `.reverse()` or slicing, use a loop to print the items of this list in **reverse order**.

```
colours = ["red", "green", "blue", "yellow"]
```

Need a hint?

- Use `range(len(colours) - 1, -1, -1)`

Expected output:

```
yellow
blue
green
red
```

Homework 6 – Remove Duplicates

🏠 Loop through the list and build a **new list** that has no duplicates. Keep the **first** time each item appears.

```
numbers = [1, 3, 2, 3, 4, 1, 5, 2]
```

Need a hint?

- Use `in` to check if the item is already in your new list

Expected output:

```
[1, 3, 2, 4, 5]
```

Homework 7 – Count Vowels in a Word

🏠 Ask the user to type a word. Loop through each letter and count how many times each vowel (`a` , `e` , `i` , `o` , `u`) appears. Print the count for each vowel.

Expected output (for `programming`):

```
a: 1
e: 0
i: 1
o: 1
u: 0
```

Homework 8 – Sorted Leaderboard

🏠 Find the **top 3** players by score and print their names and scores in order.

```
players = [("Alice", 42), ("Bruno", 91),
           ("Carla", 67), ("Dina", 88),
           ("Erik", 55)]
```

Need a hint?

- Sort the list by score first. You can use `key=lambda p: p[1]`.

Expected output:

```
1. Bruno – 91
2. Dina  – 88
3. Carla – 67
```

WEEK 6 — LISTS & TUPLES

Homework 9 — Build a Shopping List

🏠 Keep asking the user to type an item. Add each item to a list. When they type `done`, stop and print the full list.

Expected output:

```
Add item (or 'done' to finish): milk
Add item (or 'done' to finish): bread
Add item (or 'done' to finish): done
Your list: ['milk', 'bread']
```

Homework 10 – Two-List Comparison

🏠 Print only the names that appear in **both** lists.

```
list_a = ["Alice", "Bruno", "Carla", "Dina"]
list_b = ["Bruno", "Erik", "Dina", "Fatima"]
```

Expected output:

```
Bruno
Dina
```



See you next week!